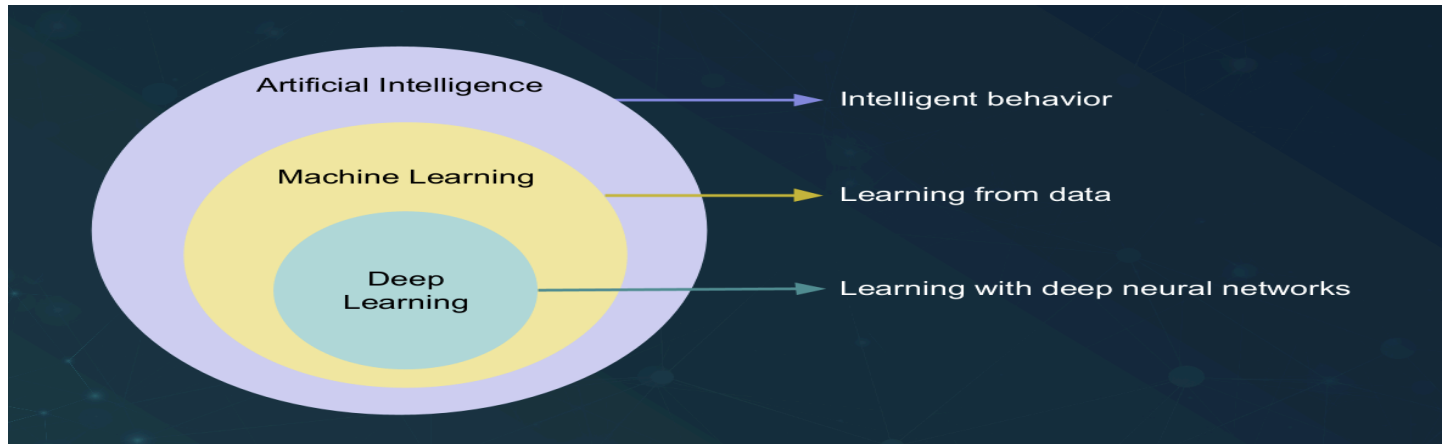


1) Artificial Intelligence vs Machine Learning vs Deep Learning

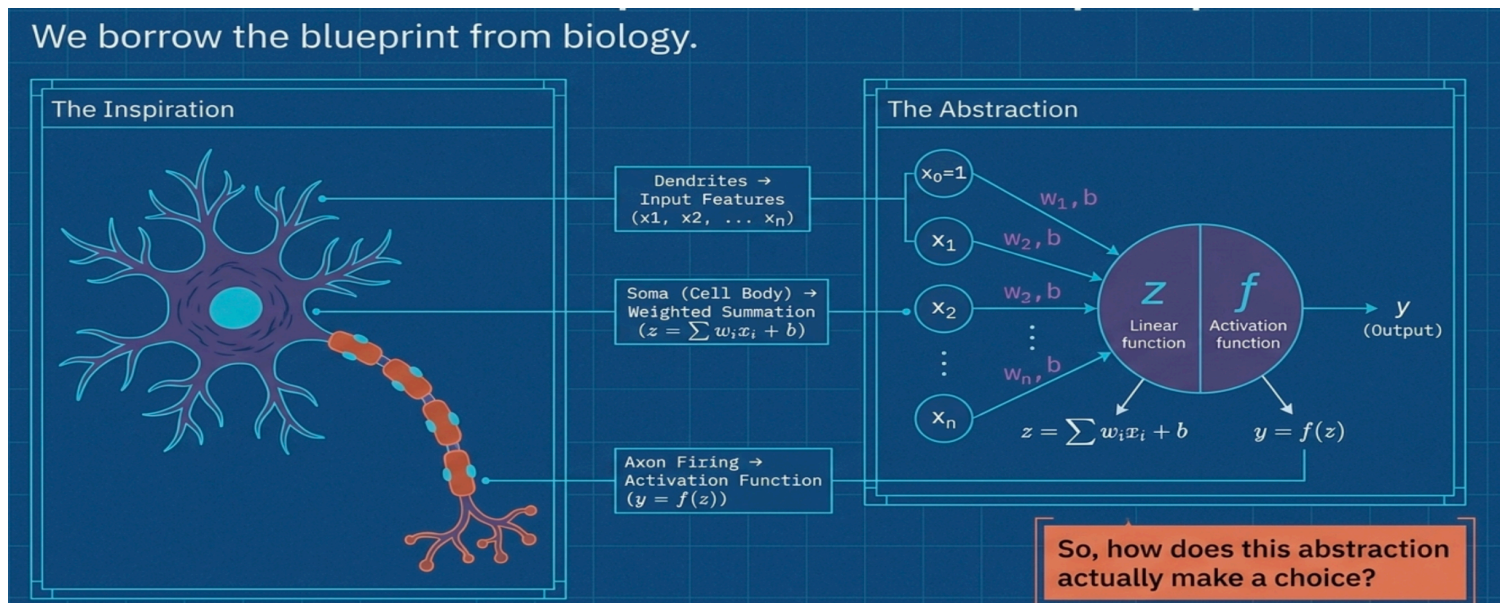
- Artificial Intelligence (AI): A broad field that focuses on intelligent behavior. Any system that mimics human intelligence to perform tasks and can improve itself based on the information it collects
- Machine Learning (ML): Data-Driven AI, i.e. focuses on learning patterns from data. It's a subset of AI that learns a mathematical function $f(x, \theta)$ from data, where parameters (θ) are learned through optimization
- Deep Learning (DL): ML with Deep Neural Networks, i.e., focuses on learning with deep neural networks. It's a specialized subset of ML where the model consists of many layers of "neurons". In traditional ML, we often have to manually "engineer" features (identifying which parts of the data are important). In DL, the model learns both features and predictions jointly. For example, early layers might automatically learn to detect edges, while deeper layers combine them to recognize complex objects like faces



2) What exactly is a neuron?

- An artificial neuron, inspired from biology, is a fundamental building block of a neural network, designed as a mathematical function that maps input signals to an output
- It has four primary components
 - Input Features (x_i): These represent the incoming data / features / signals
 - Weights (w_i): These parameters represent the importance or strength of influence of each feature
 - Bias (b): This allows the neuron to generate a baseline output even when input features are zero, providing flexibility in shifting the decision boundary
 - Activation Function (f): This is a nonlinear transformation applied to the integrated features. It determines whether and how the neuron "fires" an output (y) based on a linear combination of its inputs ($z = \sum w_i * x_i + b$)
 - Common examples include the Sigmoid (for probabilities) and ReLU (for hidden layers)

We borrow the blueprint from biology.

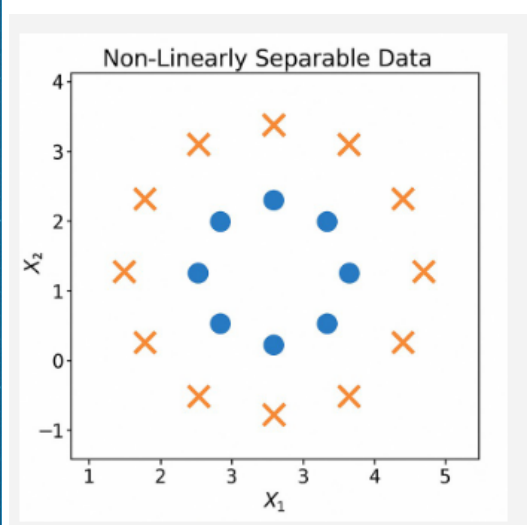
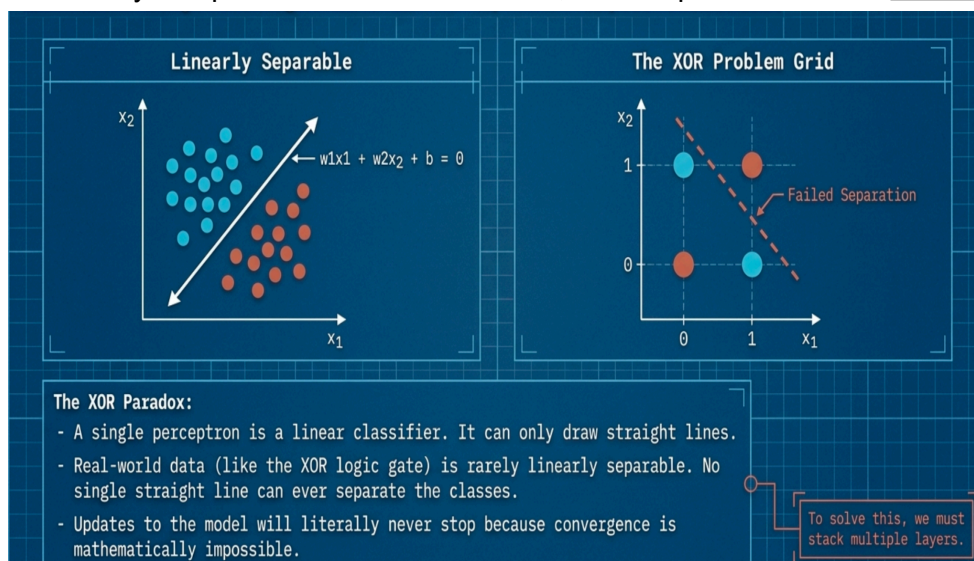


3) How does a neuron make decisions? Why do we need activation functions?

- The neuron receives multiple input features (x_i).
- Each input is multiplied by a weight (w_i), which represents that signal's importance or strength of influence.
- The neuron then adds a bias (b) to compute a weighted sum ($z = \sum w_i * x_i + b$).
- Without an activation function
 - A neuron simply computes a weighted sum of its inputs. ($z = \sum w_i * x_i + b$)
 - A neuron doesn't really "decide" anything. It only scales and combines inputs
 - Hence, the computed value (z) is linear combinations of the inputs.
 - Stacking multiple neurons (linear function) just creates one big linear function.
- Activation function is the one that converts the weighted sum into meaningful output (decision).
 - Example: sum is converted into a probability or an activation level. A final threshold (for example, 0.5) can then be used to decide Pass or Fail.
- The activation function introduces nonlinearity, allowing the neuron to model complex relationships
 - Non-linear activations allow the network to bend and warp the decision boundary, enabling it to learn the complex curved boundaries required to understand real-world data
- **A single neuron can only learn one linear rule, how to solve more complex problems?**

4) What is a neural network? Why do we need it?

- A single neuron is essentially a simple linear classifier. A single straight line cannot separate complex realities. A classic example is modelling "XOR".
 - When you plot the XOR data points on a 2D plane, the "1"s and "0"s are positioned diagonally across from each other.
 - Because of this arrangement, there is no single straight line that can separate the two classes correctly. No single line can separate the two outputs.
 - Think of a single neuron as having one knife cut. XOR needs multiple cuts.
- A neural network is formed by stacking many individual neurons together, similar to how a complex structure is built from individual Lego bricks
- Layered Structure: A standard Feed-forward network organizes neurons into three types of layers:
 - Input Layer: Receives the raw data features.
 - Hidden Layers: Intermediate layers that are "hidden" because their true values are not observed in the training set. A network with many hidden layers is called a Deep Neural Network.
 - Output Layer: Generates the final prediction
- In a standard Multi-Layer Perceptron (MLP), information flows in one direction: Input → Hidden → Output
- The true power of a neural network lies in its ability to learn hierarchical representations
 - Early layers detect simple, low-level features (e.g., edges in an image or basic tones in audio)
 - Deeper layers combine these simple features into complex abstract concepts (e.g., eyes and noses, then full faces; or phonemes, then full words)
- By increasing the width (more neurons per layer) and depth (more layers), a network can approximate almost any complex mathematical function, a concept known as the Universal Approximation Theorem



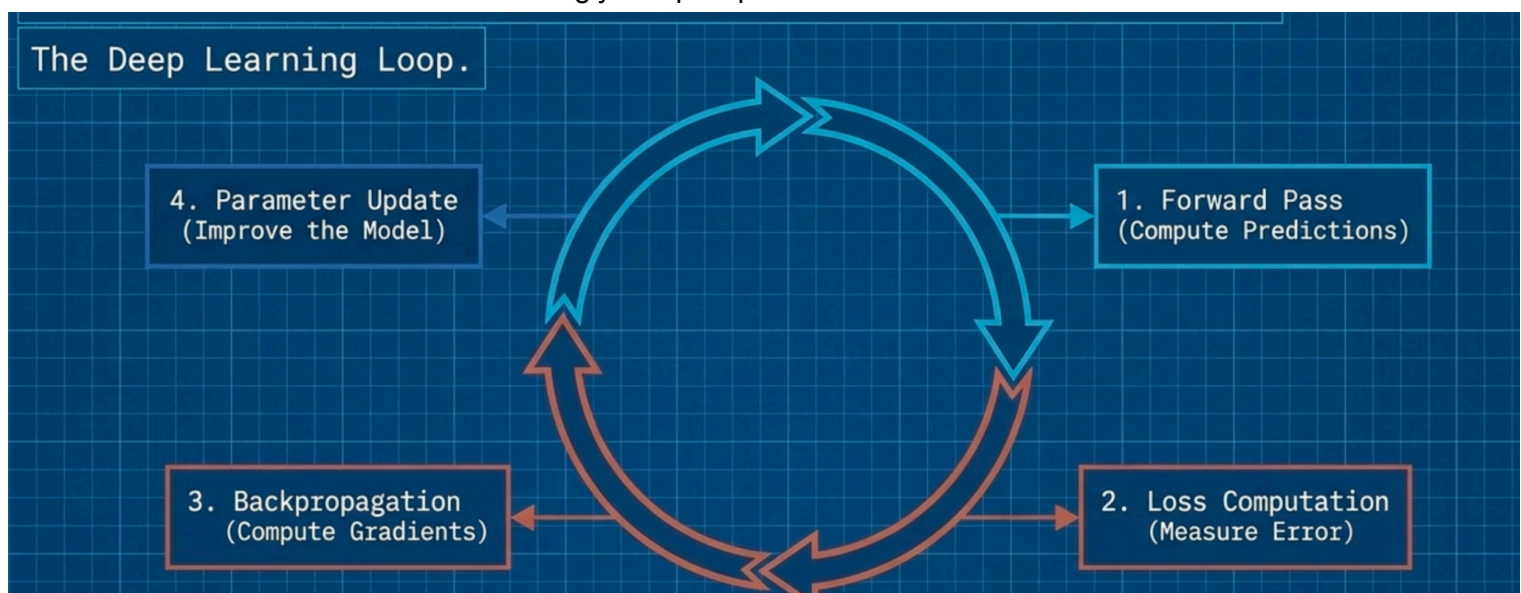
Intuition: The Neural Network layers are the total number of Lego blocks you have.

- More layers give you the size and capacity to build something massive.
- The Activation Function is the bumps on the Lego blocks that let them snap together into complex shapes.
- Without the bumps, the blocks just slide off each other, and you can only build a flat, straight pile

A neural network learns many simple rules, then combines them layer by layer into increasingly complex concepts, allowing it to solve problems that are impossible for a single neuron alone.

5) How does a neural network learn?

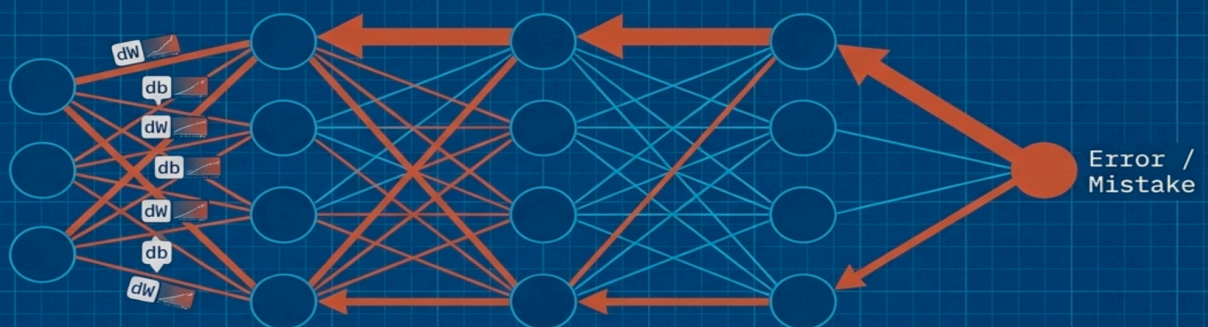
- A neural network learns through an iterative process of adjusting its learnable parameters to minimize prediction errors.
- What are the learnable parameters? What actually learns?
 - It's the weights and biases of all the neurons in a neural network.
 - Weights determine the importance of each input feature.
 - Biases shift the neuron's decision boundary.
 - That's how the neural network automatically learns which features are important.
 - The activation functions (such as ReLU and Sigmoid) remain fixed throughout training.
- The learning process consists of five steps, aka, Deep Learning Loop
 - Forward Pass: Make a Prediction. The input data is passed through the network layer by layer until it produces a prediction.
 - Loss Computation (Error): The prediction is compared with the actual (ground truth) value. A loss function measures how wrong the prediction is. The larger the loss, the further the prediction is from the correct answer.
 - Backward Pass (Compute Gradients): A gradient measures how sensitive the loss is to a parameter. Once the loss is known, the network works backwards through all the layers to determine **which parameters contributed to the loss**.
 - Parameter Update: Gradient Descent. The network adjusts its weights and biases so that the next prediction is slightly better. The goal is to minimize the loss. The most common algorithm used for this is Gradient Descent, which updates the parameters in the direction that reduces the loss.
 - Repeat: The network repeats this cycle hundred (or even thousands) of times over the training data. With each iteration, the prediction error decreases and the neural network model gradually learns the underlying patterns in the data. Hence, over many iterations, these small improvements enable the network to discover increasingly complex patterns in the data.



6) How does a neural network determine which parameters contributed to the loss?

- Blame assignment: A neural network may contain thousands or even millions of parameters. Simply knowing that the prediction was wrong is not enough, it must determine which parameters contributed the most to the loss.
- The answer is Backpropagation. Instead of stopping at the final prediction, the network works backwards through every layer, tracing the error back to every neuron.
- At each neuron, it asks a simple question: "How much did its parameter contribute to the final error?"
 - Weights that contributed more receive more blame.
 - Weights that contributed less receive less blame.
- The network uses the Chain Rule from calculus to calculate how the error flows backwards through the network. For now, think of it as a systematic way of tracing the error back through every connection assigning responsibility to each parameter (weights & bias). The result of this calculation is called the gradient.
- A gradient tells us:
 - How much a weight contributed to the error.
 - Whether increasing or decreasing that weight will reduce the error. (steepest descent)
- One way to visualize this is to look at the neural network as a graph. Forward pass is going from input to output layer and computing loss or error. Backward pass is going from loss to input layer computing gradient.

Backpropagation.



The Blame Assignment:

- A neural network contains thousands of weights and biases.
- Backpropagation computes a precise **responsibility signal** (the derivative) for every single parameter.
- Using the Chain Rule of calculus, it asks: "If I change this specific weight slightly, how much will the total error decrease?"

Now that we have assigned the blame, how do we fix the machine?

7) Once the network knows the blame, how does it fix the mistake?

- Once the gradients have been calculated, the network updates its weights using Gradient Descent.
- Imagine you're hiking in thick fog on a mountain. Your goal is to reach the lowest point in the valley. You can't see the entire mountain, but you can tell which direction slopes downhill beneath your feet. So you:
 - Take a small step downhill.
 - Stop and look around.
 - Take another small step.
 - Repeat until you reach the bottom.
- Gradient Descent is taking that small step in the right direction. If we take large steps, we risk losing stability and falling. Similarly, changing the weights too much can make the network overshoot the optimal solution.
- Gradient descent is an optimization algorithm used to minimize a function (such as error or loss) by adjusting parameters (like weights and biases) in the direction of steepest descent.

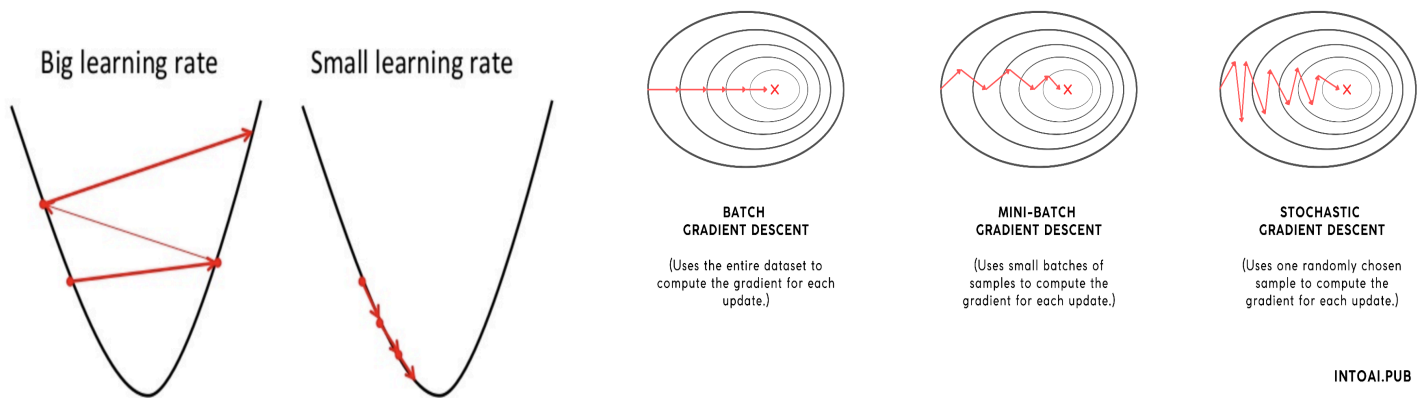
In summary

- **Gradient** = Measure the blame
- **Chain Rule** = A calculus trick used to find the rate of change for nested functions with respect to one variable
- **Backpropagation** = Determine the Gradient (blame) for each parameter using Chain Rule (calculus)
- **Gradient Descent** = Correct the mistake for each parameter (Update Parameters) Updates the parameters by taking small steps in the direction that reduces the loss.
- By repeating this Deep Learning Loop thousands of times, the network gradually improves its predictions.

8) How can we make learning faster and more stable?

- Gradient Descent gradually improves the network by updating its parameters (weights & biases).
- Here's the update rule for Gradient Descent: $\theta = \theta - \alpha * \nabla L(\theta)$, where
 - θ is parameter, L is the loss function
 - $\nabla L(\theta)$ is gradient of the loss with respect to the parameter θ (computed using Backpropagation)
 - α is the learning rate (step size). It decides how big or small the steps are to update parameter θ
- The simplest algorithm is Batch Gradient Descent, it uses the entire training dataset to calculate the gradient and update model parameters once per iteration (aka epoch).
- In practice, learning is not always smooth. Sometimes the network:
 - Learn too slowly when the learning rate is too small.
 - Overshoot or oscillate when the learning rate is too large.
 - Make little progress when gradients become very small.
 - Take unnecessary paths before reaching a good solution.
- To overcome these challenges, several improved optimization techniques have been developed.

What are we improving?	Optimizer / Technique	Main Idea	When to Use
Training Data (Batch Size)	Batch Gradient Descent (Simplest)	Uses the <u>entire training dataset</u> to compute one update.	Small datasets where stable updates are more important than speed.
	Stochastic Gradient Descent (SGD)	Updates parameters after <u>every training example</u> .	Very large datasets; faster but noisier learning.
	Mini-batch Gradient Descent	Updates parameters using a <u>small batch</u> of examples.	Default choice for most deep learning applications because it balances speed and stability.
Parameter Update Strategy	Gradient Clipping	Instead of applying the raw gradient, we limit its size before upgrading parameters	A large gradient can ruin an otherwise stable training process.
	Momentum	Uses <u>previous updates</u> to <u>build momentum</u> and reduce oscillations. Momentum gradient descent maintains an exponential moving average of past gradients to update the parameters	When training is slow or bounces back and forth across the optimum. It converges faster in a consistent direction and slows down on zig-zag.
Learning Rate	Learning Rate Decay	Continuously reduce the learning rate by a fixed factor.	Three methods: Step, Exponential, Cosine Annealing
	RMSProp	Automatically adapts the learning rate for each parameter. The learning rate is inversely proportional to RMS. RMS = Root Mean Square, and is directly proportional to gradient^2	When different parameters learn at different speeds or gradients vary widely. If the loss curve is flat, it takes steep jumps. If the loss curve is sharp it takes small steps.
Mix	Adam (Adaptive Moment Estimation)	Combines <u>Momentum</u> and <u>RMSProp</u> for adaptive and stable learning.	General-purpose optimizer and the default choice for most modern neural networks.



9) How do you evaluate the neural network performance on the test set (unseen data)?

- A neural network is trained using the training dataset. However, our goal is to make accurate predictions on new, unseen data (aka test dataset).
- Dataset Split
 - Training Set: Used to learn the model parameters (weights and biases).
 - Validation Set: Used to tune hyperparameters and select the best model during training.
 - Test Set: Used only once at the end to measure how well the model generalizes to unseen data.
- Common Evaluation metrics
 - Classification: For tasks such as spam detection or image classification, common metrics include: Accuracy, Precision, Recall, F1-Score, Confusion Matrix, Regression
 - Regression: For predicting continuous values, common metrics include: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), R^2 Score
- In classification, we have binary and multi-class classification. We calculate the confusion matrix for each class. Here's an example for binary classification

		Predicted			
		Species _k	Other sp.		
Observed	Species _k	True Positive	False Negative		
	Other sp.	False Positive	True Negative		

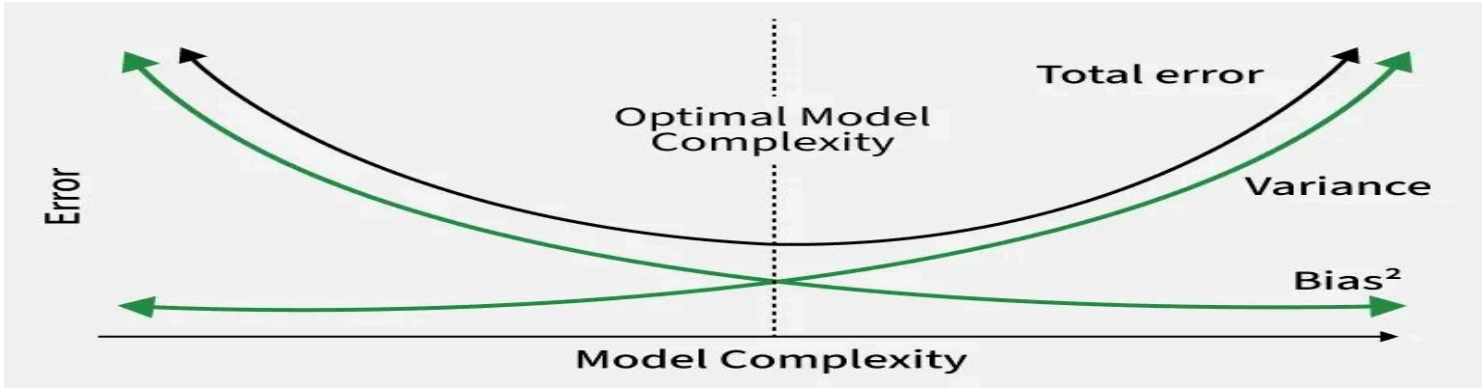
	Accuracy	=	$\frac{TP + TN}{TP + TN + FP + FN}$
	Specificity	=	$\frac{TN}{TN + FP}$
	Precision	=	$\frac{TP}{TP + FP}$
	Recall	=	$\frac{TP}{TP + FN}$

- Most often, accuracy is often not enough. Especially in cases where False positive or False negative is costly. Consider a fraud detection dataset contains:
 - 990 legitimate transactions and 10 fraudulent transactions
 - You detect all legitimate transactions, and have accuracy of 99%
 - Despite the high accuracy, the model completely fails to detect fraud.
 - Missing a fraudulent transaction is usually very costly. Recall measures how many actual frauds we successfully detected. A few false alarms are acceptable because they can be manually reviewed.
- Take away:
 - The best evaluation metric is determined by the business problem, not by the ML algorithm.
 - A good model should perform well on both training and test dataset.

10) If the model keeps getting better during training, why does it sometimes perform poorly on unseen data

- A neural network learns by minimizing the loss on the training dataset. However, our goal is not to build a model that only performs well on the training data.
- Instead, we want a model that can make accurate predictions on new, unseen data. This ability is called generalization.
- A neural network that “memorizes” the training data often performs poorly on the test set.
- These three outcomes are often explained based on the model's performance on training and test dataset.

Model	Training Performance	Test Performance	Interpretation (Bias-Variance Tradeoff)
Underfitting	Poor	Poor	The model has <u>not learned</u> enough and it's too simple. (Has high bias: oversimplified assumptions about the data)
Overfitting	Excellent	Poor	The model is complex and has <u>memorized</u> the training data (Has high variance: pays too much attention to the noise)
Good Fit	Good	Good	The model has learned the underlying patterns. (balances bias and variance)



11) What factors affect the neural network performance?

- Data Quality and Quantity
- Model Architecture
- Activation function for each Layer
- Loss function
- Optimizers
- Hyperparameters & Tuning Methods
- Parameter Initialization
- Generalisation Techniques

12) How to initialize parameters?

- Before training begins, the network must assign initial values to all weights and biases. These values influence magnitudes of activation and gradient and overall stability of learning
- If all neurons start with the same values, they will learn the same thing and become identical. The network loses its ability to learn diverse features. Instead, weights are initialized with small random values.
- Bad initial values can cause failure to learn due to Slow training, Exploding gradients or Vanishing gradients
- Common initialization strategies include:
 - Zero: Sets all starting values to zero. This fails because network learns nothing new
 - Random: Initialize weights with small random values
 - Xavier (Glorot): Scales initial weights based on the number of input and output connections to keep activations and gradients stable; commonly used with Sigmoid and Tanh.
 - He: A variation of Xavier that uses larger initial weights to compensate for neurons that become inactive in ReLU networks; the default choice for ReLU-based models.

13) How to choose the right Activation function for each layer?

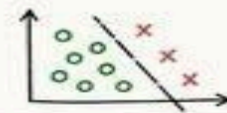
All Activation Functions at a Glance



Activation	Strengths	Limitations	Typical Use
Sigmoid	Probabilistic output	Saturation, slow ✓	Binary output ✓
Tanh	Zero-centered	Saturation ✓	Shallow hidden layers ✓
ReLU	Fast, stable	Dead neurons	Default - hidden layers ✓
PReLU	Avoids dead neurons	Extra hyper-parameter	Deep hidden layers
Softmax	Multi-class probabilities	Only for output	Multi-class output

14) How to choose the right Loss function?

10 Most Common Loss Functions in Machine Learning



Loss Function Name	Description	Function
--------------------	-------------	----------

Regression Loss Functions

Mean Bias Error	Captures average bias in prediction. But is rarely used for training.	$\mathcal{L}_{MBE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))$
Mean Absolute Error	Measures absolute average bias in prediction. Also called L1 Loss.	$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N y_i - f(x_i) $
Mean Squared Error	Average squared distance between actual and predicted. Also called L2 Loss.	$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$
Root Mean Squared Error	Square root of MSE. Loss and dependent variable have same units.	$\mathcal{L}_{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2}$
Huber Loss	A combination of MSE and MAE. Controlled by a threshold parameter δ .	$\mathcal{L}_{Huber}(\delta) = \begin{cases} \frac{1}{2} (y_i - f(x_i))^2 & : y_i - f(x_i) \leq \delta \\ \delta (y_i - f(x_i) - \frac{1}{2} \delta) & : \text{otherwise} \end{cases}$
Log Cosh Loss	Smooth alternative to MAE. Less sensitive to outliers than MSE but computationally expensive.	$\mathcal{L}_{LogCosh} = \frac{1}{N} \sum_{i=1}^N \log(\cosh(f(x_i) - y_i))$

Classification Loss Functions (Binary + Multi-class)

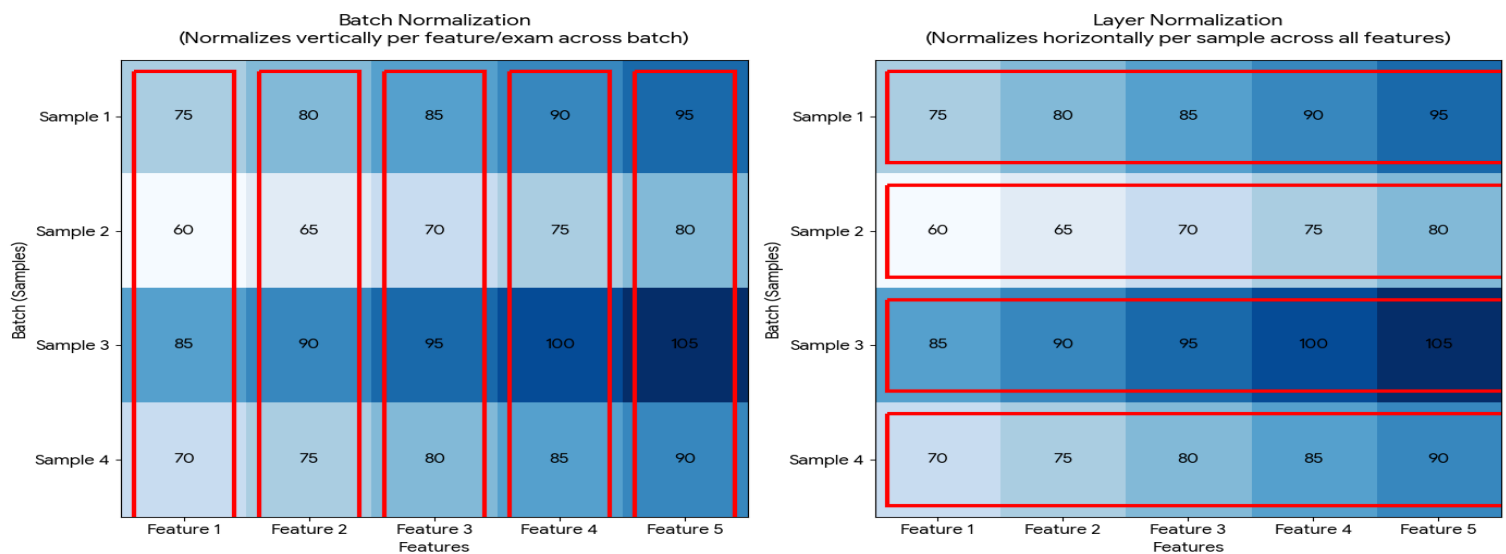
Binary Cross Entropy (BCE)	Loss function for binary classification tasks.	$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(x_i)) + (1 - y_i) \cdot \log(1 - p(x_i))$
Hinge Loss	Penalizes wrong and right (but less confident) predictions. Commonly used in SVMs.	$\mathcal{L}_{Hinge} = \max(0, 1 - (f(x) \cdot y))$
Cross Entropy Loss	Extension of BCE loss to multi-class classification.	$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \log(f(x_i)_j)$ N: samples, M: classes
KL Divergence	Minimizes the divergence between predicted and true probability distribution	$\mathcal{L}_{KL} = \sum_{i=1}^N y_i \cdot \log\left(\frac{y_i}{f(x_i)}\right)$

15) How can we prevent the neural network from memorizing the training data? Generalization Techniques

Generalization is a model's ability to perform accurately on unseen data outside its training set.

- Early Stopping: Sometimes the model starts memorizing simply because it trains for too long.
- Regularization: Adds a constraint or penalty term to the loss function.
 - Dropout: Instead of allowing every neuron to participate in every training step, dropout randomly turns off a fraction of neurons during training. Prevent neurons from becoming overly dependent on one another.
 - L1 and L2 Regularization: Sometimes the model becomes unnecessarily complex by assigning very large weights. L1 and L2 Regularization add a penalty to the loss function for large weights.
 - L1 punishes the model based on $|w|$ (absolute value). L1 creates sparse models by setting some coefficients completely to zero, acting as an automatic feature selection tool
 - L2 punishes the model on w^2 (square). The idea is simpler models usually generalize better.
- Normalization: By forcing the data at every layer to consistently have a mean of 0 and a variance of 1, normalization stabilizes the data. Because the data distributions remain stable, the model learns robust, reliable patterns instead of brittle, temporary shortcuts. Also prevents "Exploding" or "Vanishing" Gradients
 - Batch Normalization: normalizes features across the mini-batch dimension
 - Layer Normalization: normalizes across all feature dimensions for each single sample.
- Data Augmentation: Sometimes the best way to prevent memorization is to provide more diverse training data. Instead of collecting new data, we create new examples from existing ones.
 - For images, this may include: Rotating, Cropping, Flipping, Adjusting brightness

Batch vs Layer Normalization



16) How do we systematically find the best combination?

Neural network performance depends on many design choices. The challenge is that there is no formula that tells us the perfect combination beforehand. Instead, we treat model development as an experimental process.

- Hyperparameter: A hyperparameter is chosen by the practitioner before training begins.
 - Ex: Learning rate, Batch size, Number of hidden layers/Architecture, Optimizer, Dropout rate etc.
- Unlike parameters, hyperparameters are not learned by the network.
- Common Hyperparameter Tuning Approaches
 - Manual/Random Search: Adjust settings based on intuition or randomly.
 - Grid Search: Try every possible combination from a predefined set of values.
 - Bayesian Optimization: Use results from previous experiments to intelligently choose the next configuration.
- Validation Set for Hyperparameter Tuning: If we repeatedly tune using the test set, we indirectly start optimizing for it. The test set is supposed to represent truly unseen data. Hence we use a validation set.
- Tracking Experiments: using tools like TensorBoard (captures hyperparams for each run and the performance metrics. We can visualise, compare and select the best hyperparams setting)

17) What are various Neural Network Architecture?

MLP is good for tabular data. Different types of data have different structures. Here are some examples

Architecture	Why MLPs Struggle	Core Idea	Best For	Key Mechanism
CNN	Images contain local patterns (edges, corners, textures). Fully connected layers treat all pixels equally and do not explicitly model local neighborhoods.	Learn spatial patterns using small filters that slide across the image.	Images, video, computer vision	Convolution filters create feature maps by detecting patterns regardless of location. Pooling filter summarizes and shrinks the feature map. (max and mean pool) Padding is for border pixels Stride is filter's next step
RNN	Feedforward networks have no memory and process each input independently. Past inputs cannot influence future predictions. Also, the input for MLP is fixed.	Introduce memory through a hidden state that is carried forward through time.	Time series, speech, text, sequences	Recurrence : hidden state stores context from previous time steps. Sequences : Instead of a fixed input, we have sequences where input order matters.
LSTM	Standard RNNs struggle with long-term dependencies due to vanishing gradients.	Add an explicit memory cell and gates to control information flow.	Long sequences, language modeling, translation	Forget, Input, and Output Gates decide what to remember, update, and expose.
Transformer	RNNs and LSTMs process sequences sequentially and struggle to capture very long-range relationships efficiently.	Allow every token to directly attend to every other token.	LLMs, translation, modern AI systems	Attention Mechanism learns what parts of the input are most relevant.

